



Course Code: INTM591	Course Title: Predictive Analytics
Academics Task No. 1	Academics Task Type: Project
Date of Allotment: 18-04-2026	Date of submission: 24-04-2026
Name- Himanshu Jangir	Student Reg. No.: 12502533
Max. Marks: _	Section: Q5E09
Evaluation Parameters: _	Submitted to: Mr. Abhishek Choudhary

DECLARATION:

I declare that this Assignment is my individual work. I have not copied it from any other students' work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part been written for me by any other person.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor, **Mr. Abhishek Choudhary**, for his invaluable guidance and constant encouragement throughout the duration of this project. His insights into modelling and data analytics helped me navigate the complexities of both Python and SPSS Modeler with greater clarity.

I am also thankful to **Lovely Professional University (LPU)** for providing a curriculum that bridges the gap between theoretical knowledge and practical industry tools. This project has been a significant learning milestone in my MBA journey, particularly in understanding how global markets correlate. Finally, I want to thank my family and peers for their constant support and feedback, which kept me motivated to complete this work with precision.

CERTIFICATE

This is to certify that Adarsh Srivastava, a student of **MBA (Business Analytics)** at **Lovely Professional University**, has successfully completed the project titled "**Exploratory Data Analysis of Titanic: Machine Learning from Disaster**" as part of the academic coursework for the subject **PREDICTIVE ANALYTICS**.

During the project, Lovish demonstrated exceptional analytical skills by performing detailed Exploratory Data Analysis (EDA) using **Python and IBM SPSS Modeler**. He successfully identified historical market patterns, handled complex time-series data cleaning, and derived meaningful insights regarding asset correlations.

His approach to the project was structured, diligent, and met all the requirements set by the department. I wish him the very best in his future academic and professional endeavors.

Signature of Supervising Teacher

(Mr. Abhishek Choudhary)

Project Definition:

This project performs Exploratory Data Analysis (EDA) on the Titanic dataset sourced from Kaggle (<https://www.kaggle.com/c/titanic/data>). The dataset contains information about 891 passengers who were aboard the Titanic, including their demographic details, ticket class, fare, cabin, and survival status. The objective is to uncover patterns and relationships in the data that influenced passenger survival, using Python (Pandas, Matplotlib, Seaborn) and IBM SPSS Modeler.

Outcomes / Learning:

After completing this project, the following skills and insights were developed:

- Hands-on experience with data cleaning, missing value treatment, and feature encoding.
- Ability to generate descriptive statistics and interpret distributions using Python and SPSS Modeler.
- Visualization of survival patterns across gender, passenger class, age, and fare using charts and graphs.
- Understanding of how to use SPSS Modeler nodes (Data Audit, Distribution, Histogram) for visual EDA.

Required Tools:

- Python 3.x — with libraries: Pandas, NumPy, Matplotlib, Seaborn, Missingno
- IBM SPSS Modeler 18.x — for stream-based EDA using Data Audit, Distribution, and Plot nodes
- Kaggle — dataset source (Titanic: Machine Learning from Disaster)
- Jupyter Notebook — for Python-based analysis and visualization

Working:

The project involves loading the Titanic CSV dataset, performing initial exploration (shape, data types, null counts), cleaning missing values in 'Age', 'Cabin', and 'Embarked', and then conducting visual analysis. In Python, charts such as survival count plots, age histograms, and correlation heatmaps are generated. In SPSS Modeler, a stream is built using Source → Data Audit → Distribution/Plot nodes to replicate the same analysis visually.

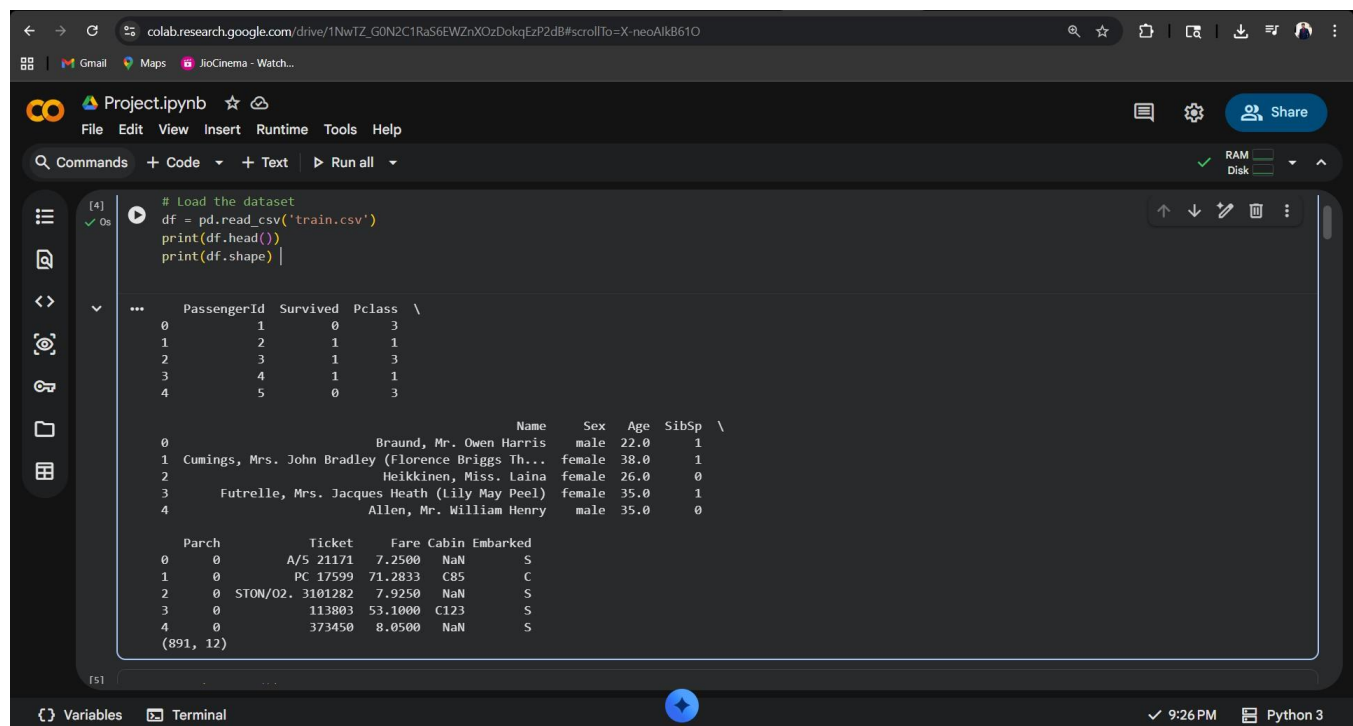
Step 1: Importing Libraries and Loading the Dataset (Python)

The first step is to import all necessary Python libraries and load the Titanic CSV dataset using Pandas. The dataset is read from the local file system after downloading from Kaggle. An initial preview of the first five rows is displayed using `df.head()` to understand the structure.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load the dataset
df = pd.read_csv('train.csv')
print(df.head())
print(df.shape)
```

Output-



```
[4]: # Load the dataset
df = pd.read_csv('train.csv')
print(df.head())
print(df.shape)
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp
0	1	0	3			
1	2	1	1			
2	3	1	3			
3	4	1	1			
4	5	0	3			

	Name	Sex	Age	SibSp
0	Braund, Mr. Owen Harris	male	22.0	1
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1
2	Heikkinen, Miss. Laina	female	26.0	0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1
4	Allen, Mr. William Henry	male	35.0	0

Parch	Ticket	Fare	Cabin	Embarked	
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

(891, 12)

Step 2: Understanding the Data — Shape, Data Types, and Basic Statistics

In this step, the dataset dimensions, column data types, and descriptive statistics are examined. The dataset has 891 rows and 12 columns. Numerical statistics such as mean, median, and standard deviation are computed to understand distributions, particularly for Age (mean ≈ 29.7) and Fare.

```
print(df.info())      # Column types & non-null counts
print(df.describe())  # Descriptive statistics
print(df.isnull().sum()) # Missing value count per column
```

Column	Data Type	Non-Null Count	Missing Values
PassengerId	int64	891	0
Survived	int64	891	0
Pclass	int64	891	0
Name	object	891	0
Sex	object	891	0
Age	float64	714	177
SibSp	int64	891	0
Parch	int64	891	0
Ticket	object	891	0
Fare	float64	891	0
Cabin	object	204	687
Embarked	object	889	2

Output-

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None

```

	PassengerId	Survived	Pclass	Age	SibSp
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

The screenshot shows a Google Colab notebook interface. The main area displays a summary of a dataset. The summary includes statistics for 'Survived' and 'Embarked' columns, and a list of variables with their counts and data types.

Statistic	Survived	Embarked
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Variable	Count	Data Type
PassengerId	0	
Survived	0	
Pclass	0	
Name	0	
Sex	0	
Age	177	
SibSp	0	
Parch	0	
Ticket	0	
Fare	0	
Cabin	687	
Embarked	2	

dtype: int64

Step 3: Handling Missing Values

Three columns contain missing values: Age (177), Cabin (687), and Embarked (2). Age is imputed with the median value to reduce the effect of outliers. Cabin is dropped due to excessive missingness (77%). The two missing Embarked rows are filled with the mode value ('S').

```
# Fill Age with median
```

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

```
# Drop Cabin column (too many nulls)
```

```
df.drop(columns=['Cabin'], inplace=True)
```

```
# Fill Embarked with mode
```

```
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
print(df.isnull().sum())
```

Output-

```
colab.research.google.com/drive/1NwTZ_G0N2C1RaS6EWZnXOzDokqzP2dB#scrollTo=daHcBVDfod

Project.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM
Disk

PassengerId 0
Survived 0
Pclass 0
Name 0
Sex 0
Age 0
SibSp 0
Parch 0
Ticket 0
Fare 0
Embarked 0
dtype: int64
/tmp/ipykernel_1386/4031361287.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead,

df['Age'].fillna(df['Age'].median(), inplace=True)
/tmp/ipykernel_1386/4031361287.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead,

df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

Step 4: Survival Count Visualization

A count plot is created to show the distribution of survivors vs. non-survivors. Out of 891 passengers, 342 survived (38.4%) and 549 did not survive (61.6%). This imbalance is important context for subsequent analysis.

```
sns.countplot(x='Survived', data=df, palette='Set2')
plt.title('Survival Count (0 = Did Not Survive, 1 = Survived)')
plt.xlabel('Survived')
plt.ylabel('Count')
plt.show()
```

Output-

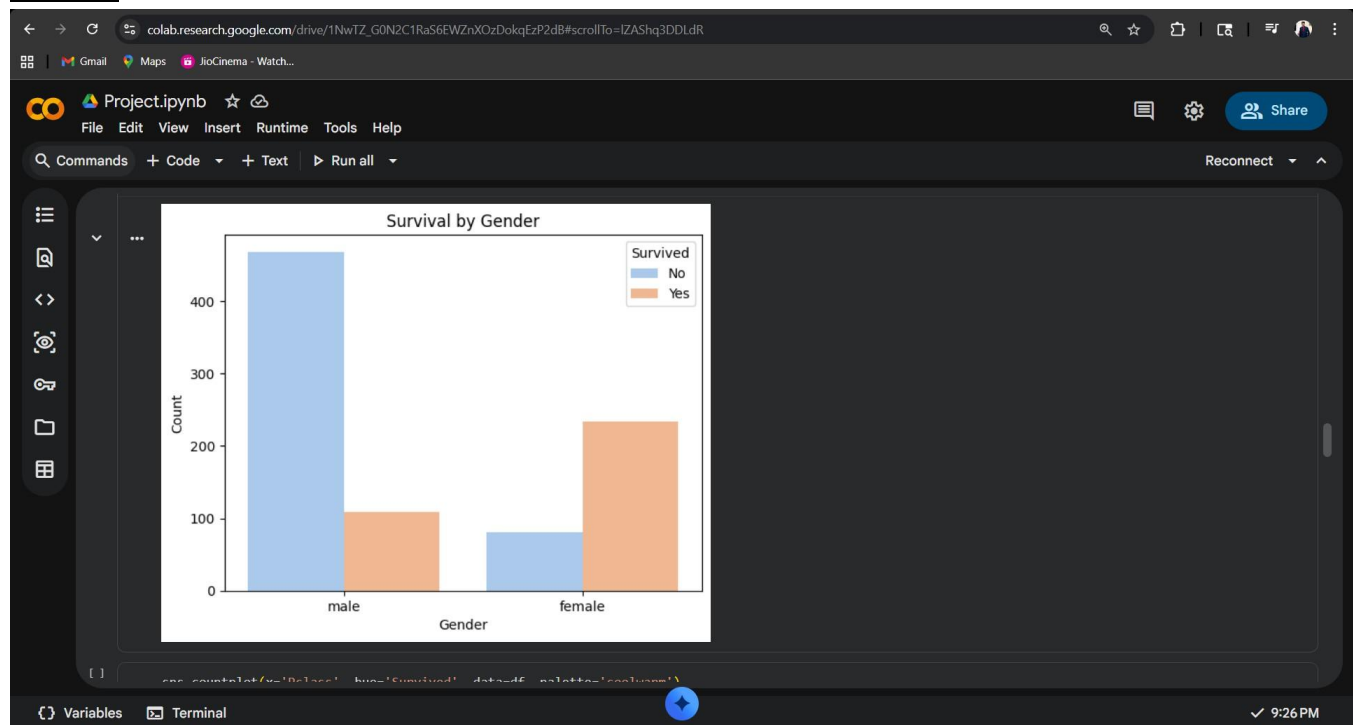


Step 5: Survival by Gender

The survival rate is broken down by gender using a grouped count plot. Female passengers had a significantly higher survival rate (~74%) compared to males (~19%), consistent with the 'women and children first' evacuation policy.

```
sns.countplot(x='Sex', hue='Survived', data=df, palette='pastel')
plt.title('Survival by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```

Output-

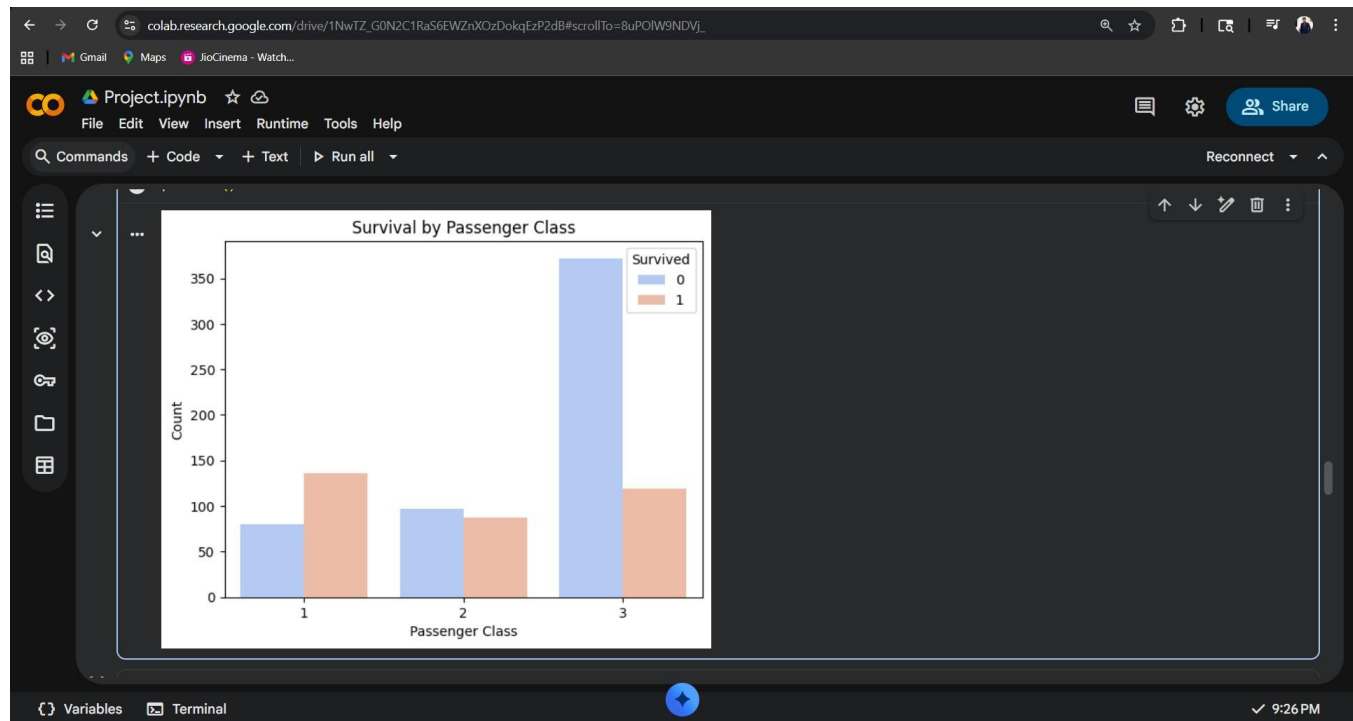


Step 6: Survival by Passenger Class (Pclass)

This step analyses how ticket class (1st, 2nd, 3rd) influenced survival. First-class passengers had a survival rate of approximately 63%, second class ~47%, and third class only ~24%. Wealthier passengers with higher-class tickets had better access to lifeboats.

```
sns.countplot(x='Pclass', hue='Survived', data=df, palette='coolwarm')
plt.title('Survival by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Count')
plt.show()
```

Output-

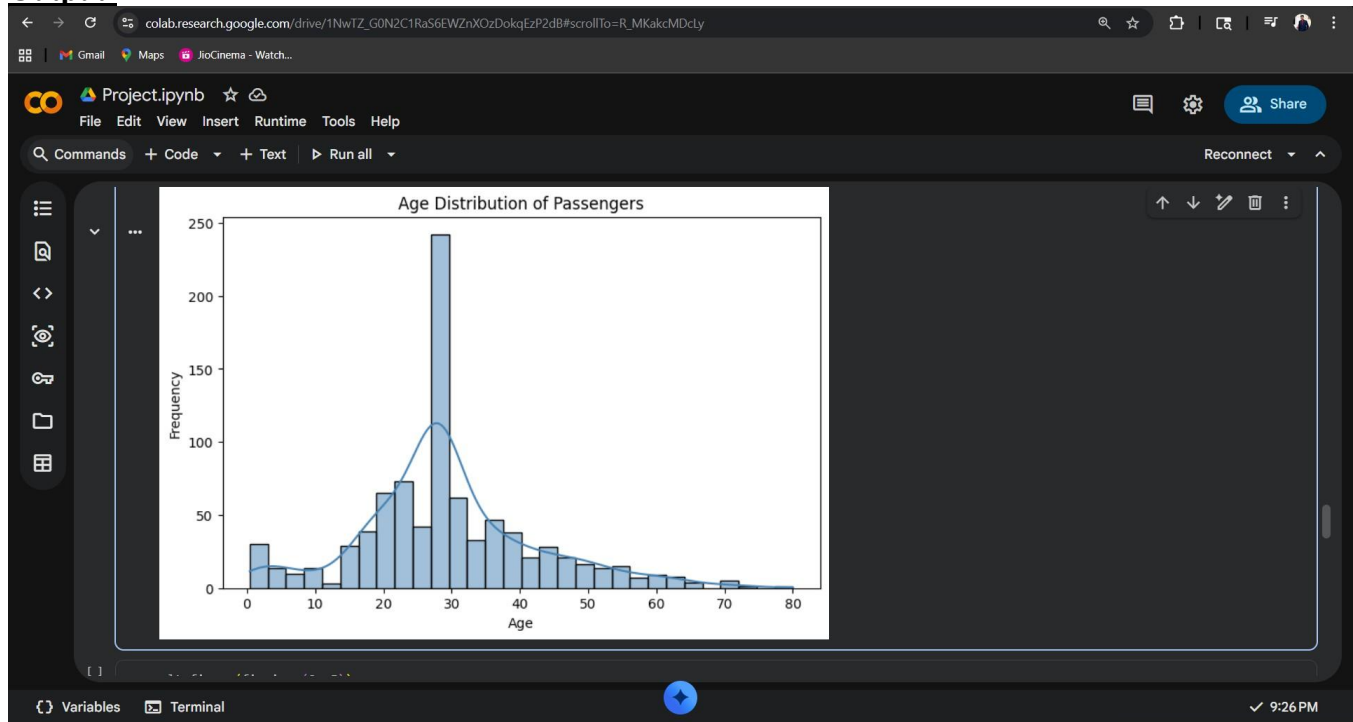


Step 7: Age Distribution Histogram

A histogram of the Age column reveals a roughly right-skewed distribution with most passengers between 20–40 years old. A KDE (Kernel Density Estimation) curve is overlaid to show the age density. Children under 10 had relatively higher survival rates.

```
plt.figure(figsize=(8, 5))
sns.histplot(df['Age'], bins=30, kde=True, color='steelblue')
plt.title('Age Distribution of Passengers')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```

Output-

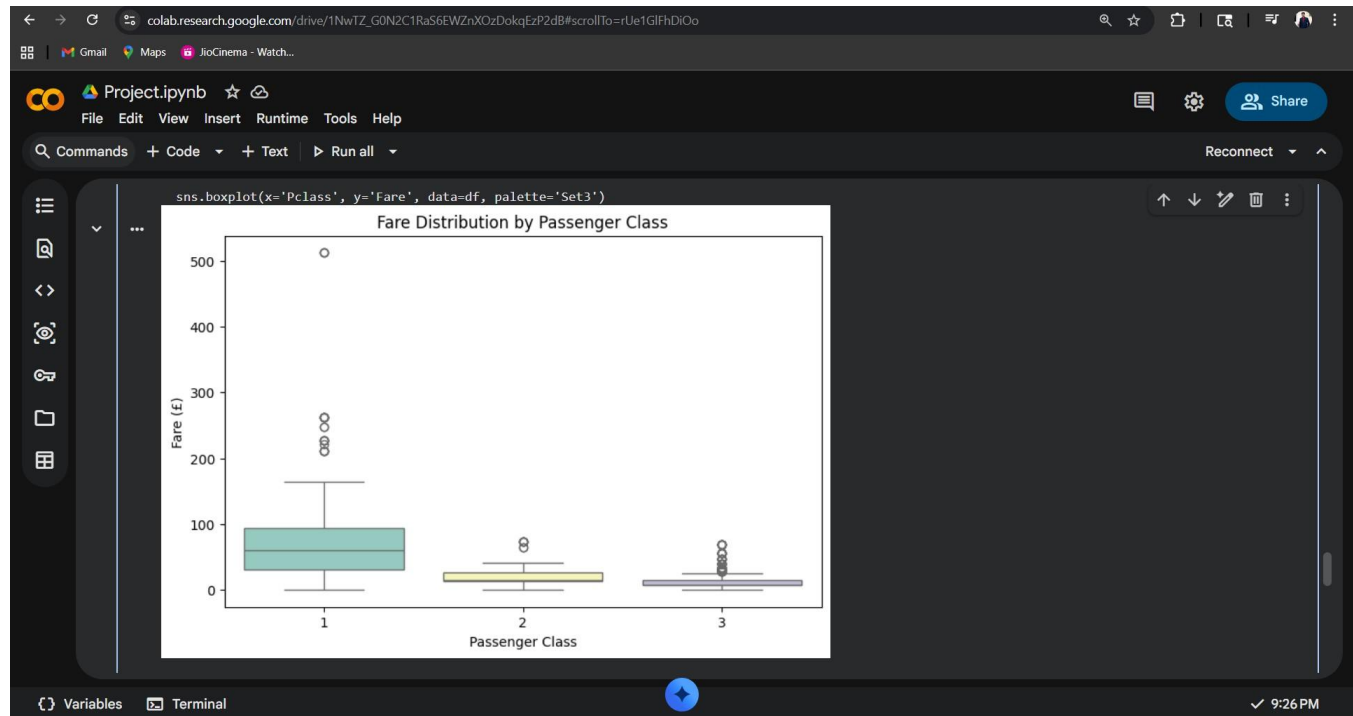


Step 8: Fare Distribution and Boxplot by Class

A boxplot of Fare grouped by passenger class shows that 1st class passengers paid significantly higher fares with more outliers. The median fare for 3rd class was around £8, while for 1st class it was approximately £60. Fare is positively correlated with survival.

```
plt.figure(figsize=(8, 5))
sns.boxplot(x='Pclass', y='Fare', data=df, palette='Set3')
plt.title('Fare Distribution by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Fare (£)')
plt.show()
```

Output-



Step 9: Correlation Heatmap

A heatmap of the correlation matrix reveals relationships between numerical variables. 'Survived' shows a positive correlation with Fare (0.26) and a negative correlation with Pclass (-0.34), confirming that higher class and higher fare improved survival odds.

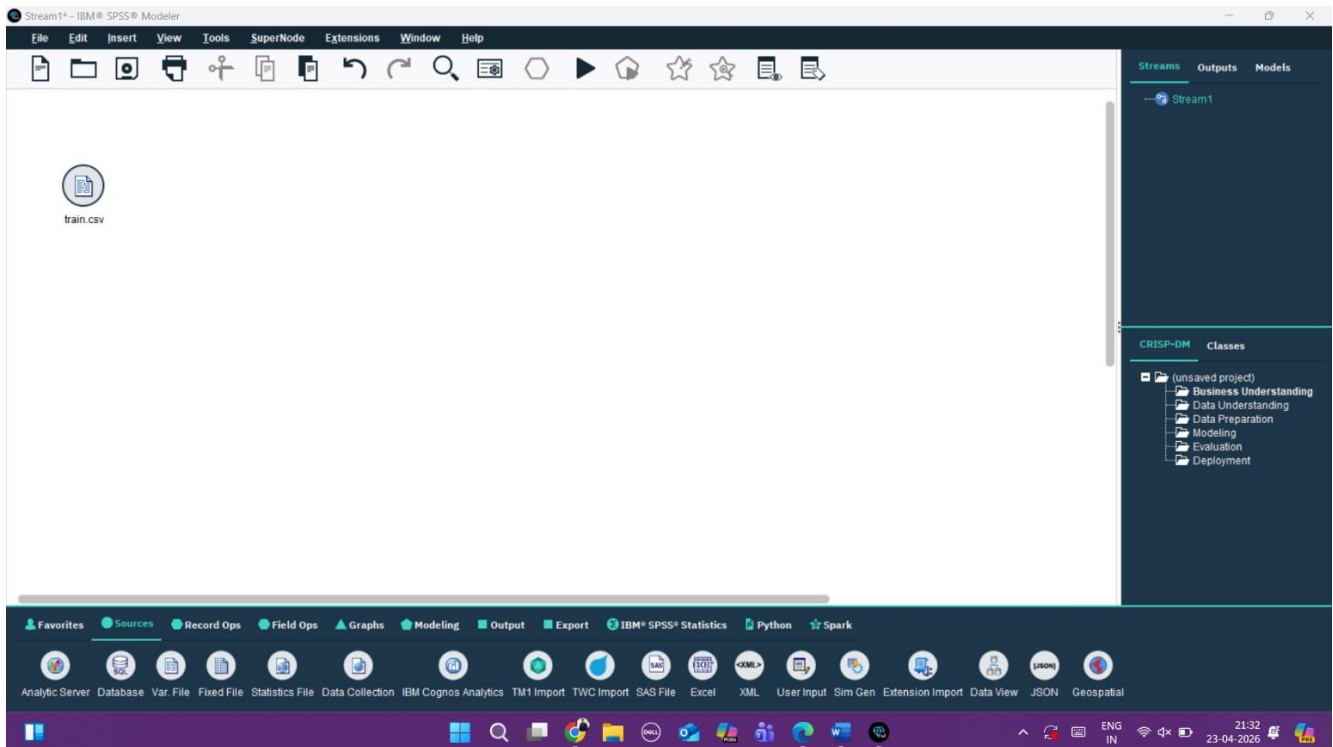
```
plt.figure(figsize=(8, 6))
corr = df[['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare']].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()
```

Output-



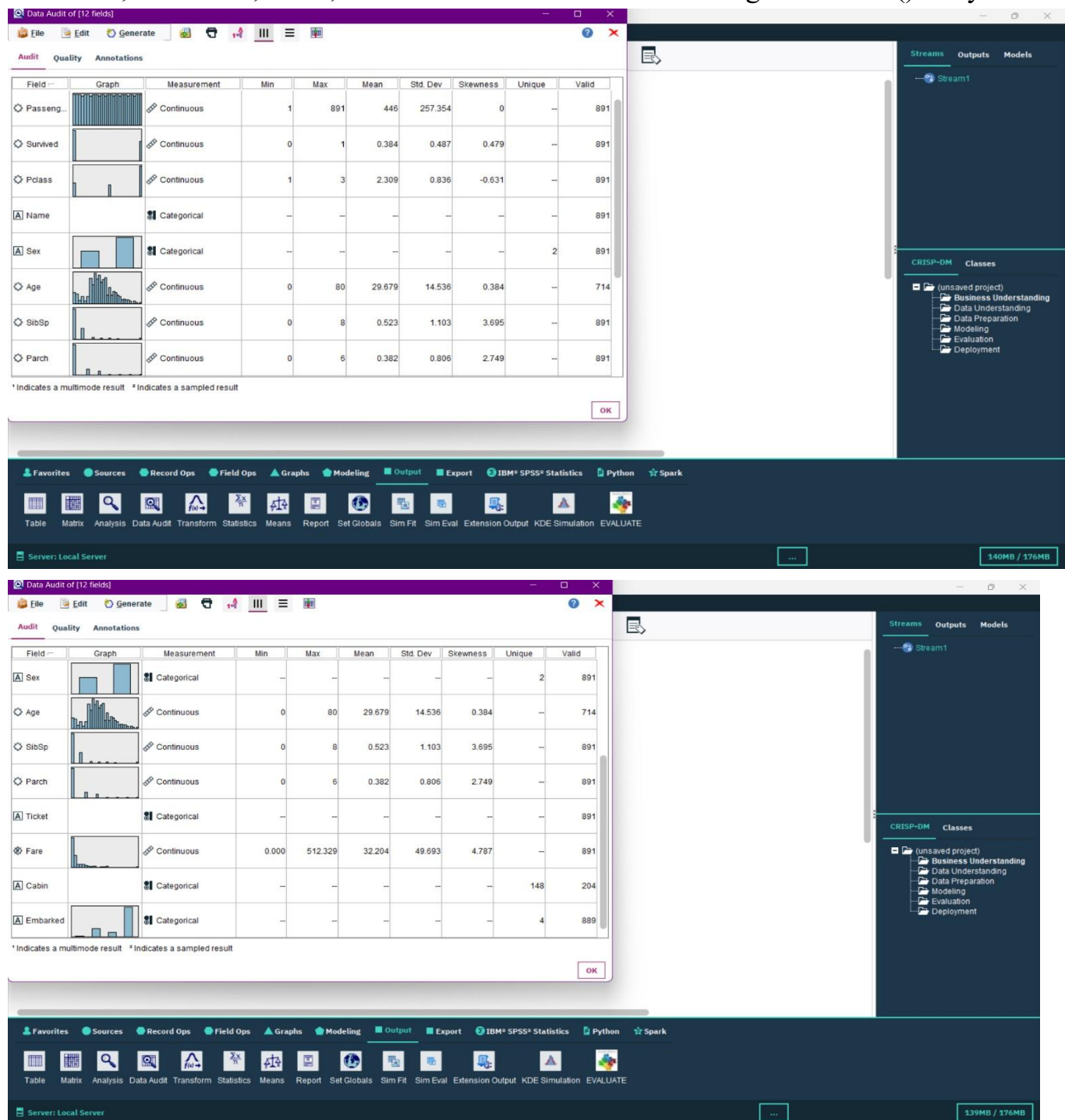
Step 10: Setting Up SPSS Modeler — Importing the Dataset

In IBM SPSS Modeler, a new stream is created. A 'Variable File' source node is added and configured to point to the train.csv file downloaded from Kaggle. The node is run to load 891 records into the stream for further analysis.



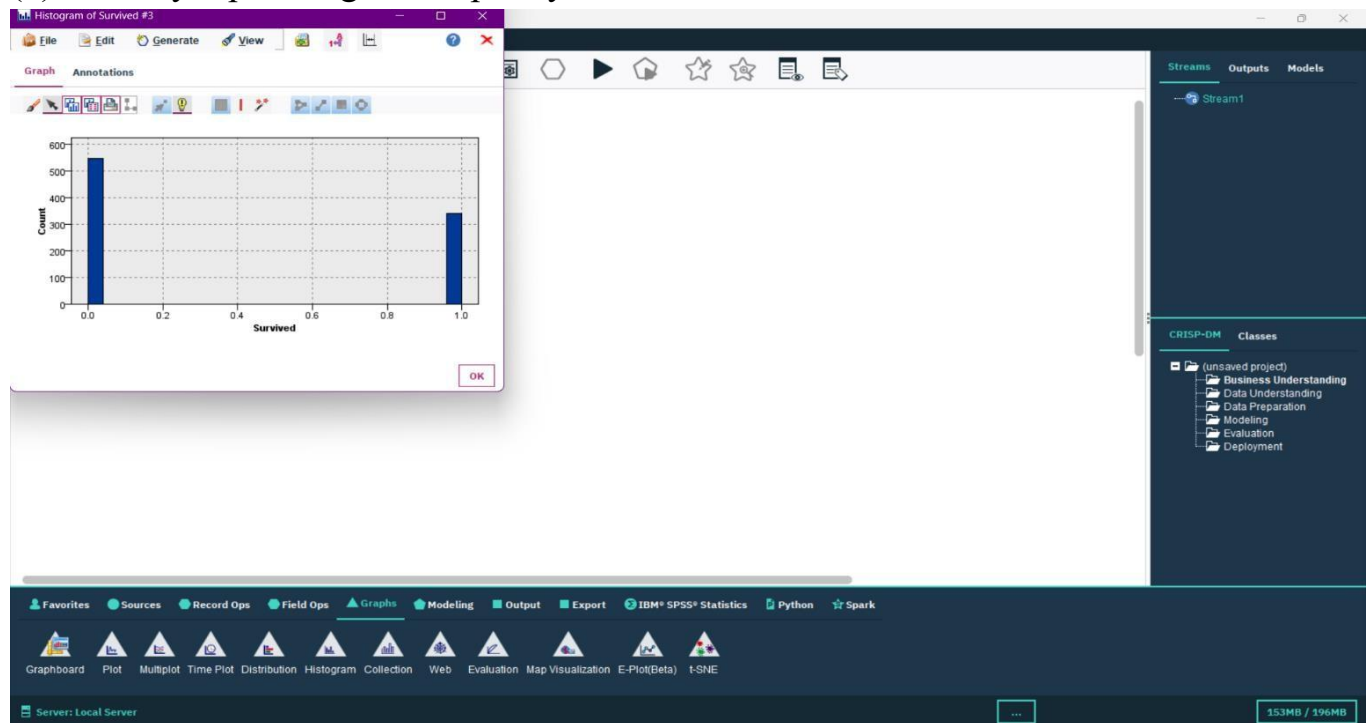
Step 11: Data Audit Node in SPSS Modeler

A 'Data Audit' node is connected to the source node and executed. This node provides a comprehensive summary of each column including count, percentage of missing values, minimum, maximum, mean, and standard deviation — mirroring `df.describe()` in Python.



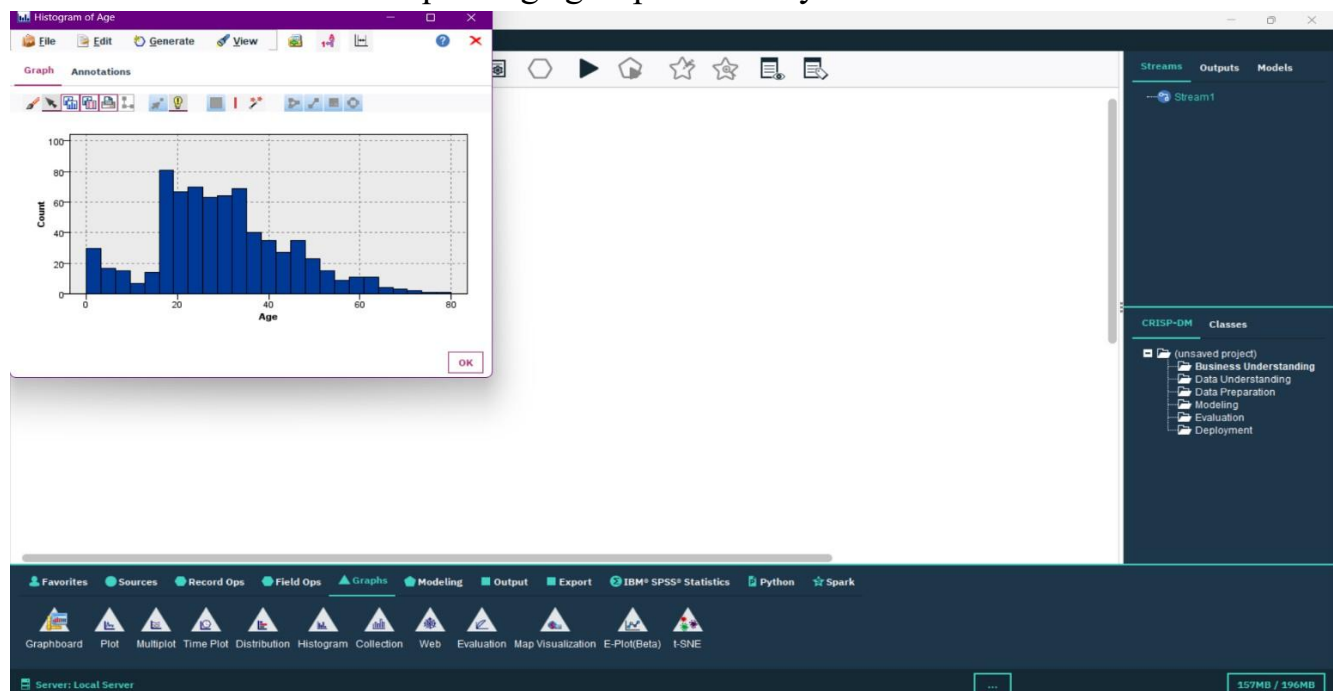
Step 12: Histogram Node — Survival Count in SPSS Modeler

A 'Histogram' node is connected and configured with 'Survived' as the field. When executed, it produces a bar chart showing the count of survivors (1) and non-survivors (0), visually replicating the Step 4 Python chart within the SPSS environment.



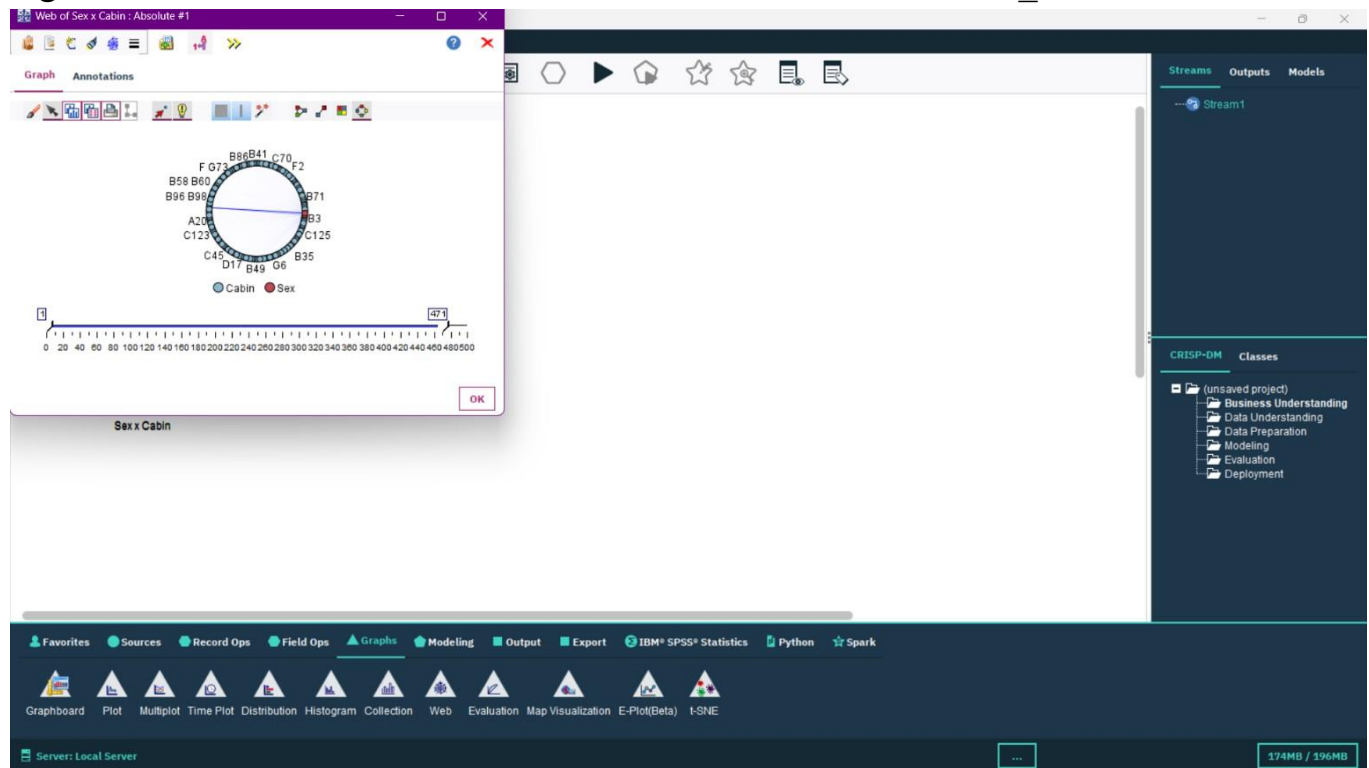
Step 13: Histogram Node — Age Distribution in SPSS Modeler

A 'Histogram' plot node is added and set to visualize the Age column distribution. The resulting histogram matches the Python-generated plot, confirming consistency between the two tools. The most frequent age group is 20–35 years.



Step 14: Web Graph Node — Survival by Gender & Class in SPSS Modeler

A 'Multiplot' or 'Web' graph node is configured to cross-tabulate Survived with Sex and Pclass. The output confirms the Python findings: females and first-class passengers had higher survival rates. The SPSS Modeler stream is saved as 'Titanic_EDA.str'.



Key Insights & Findings:

- Gender was the strongest survival predictor: 74% of females survived vs. only 19% of males.
- Passenger class strongly correlated with survival — 1st class passengers had the highest survival rate (63%).
- Age: Children (under 10) had a higher survival rate; elderly passengers were less likely to survive.
- Fare was positively correlated with survival ($r = 0.26$), as higher fares indicate higher class.
- Cabin data was too sparse (77% missing) to yield reliable insights and was removed from analysis.
- Both Python and SPSS Modeler produced consistent findings, validating the analysis across platforms.